

RTX 5090 (Blackwell) Docker 환경 구축 및 운영 가이드

작성자: 안용상

대상 장비: NVIDIA RTX 5090 (Architecture: Blackwell sm_120)

핵심 목표: JIT 컴파일 지연 없는 즉각적인 학습/추론 환경 구축 및 라이브러리 영속성 확보

참고한 자료: medium.com

1. 이 방법이 왜 필요한가? (Why)

문제 상황: "버전 불일치와 성능 저하"

- **아키텍처 불일치:** RTX 5090은 **Blackwell (sm_120)** 아키텍처를 사용합니다. 하지만 현재 `pip install torch` 로 설치되는 일반적인 PyTorch는 이전 세대인 Hopper(sm_90)까지만 바이너리를 포함하고 있습니다.
- **JIT (Just-In-Time) 컴파일 지연:** 바이너리가 없으면 PyTorch는 실행 시점에 코드를 sm_120 용으로 실시간 번역(JIT Compile)하려고 시도합니다. 이는 **초기 실행 속도를 극도로 느리게 만들거나(수 분~수 십분 소요), 아예 오류를 뿜으며 실행 불가 상태가 됩니다.**
- **환경 오염:** 호스트 OS(우분투 등)에 CUDA 12.8을 직접 깔고 버전을 맞추다 보면, 다른 프로젝트나 시스템 라이브러리와 충돌하여 "포맷밖에 답이 없는" 상황이 옵니다.

해결책: "Docker 컨테이너 격리"

- 이 방식은 **호스트 OS를 전혀 건드리지 않고**, 컨테이너 내부에만 **CUDA 12.8 + PyTorch (cu128)** 환경을 구축하여 완벽한 호환성을 보장합니다.

2. 작동 원리 (Mechanism)

이 시스템은 **3단계 레이어**로 작동합니다.

1. Layer 1: Host Driver (물리적 연결)

- 호스트 컴퓨터에는 ****NVIDIA Driver (570.xx 이상)****만 있으면 됩니다. CUDA Toolkit을 호스트에 깔 필요가 없습니다.
- Docker의 `nvidia-container-toolkit` 이 GPU를 컨테이너 내부로 "패스스루(Passthrough)" 시켜줍니다.

2. Layer 2: Docker Image (소프트웨어 본체)

- **Base Image:** `nvidia/cuda:12.8.0` 이미지가 OS 레벨의 라이브러리를 제공합니다.
- **PyTorch Wheel:** `cu128` 인덱스에서 다운로드한 PyTorch는 이미 **`sm_120` 용으로 컴파일된 실행 파일**을 가지고 있습니다. (JIT 과정 생략 → 즉시 실행)

3. Layer 3: Volume Mount (데이터 연결)

- 사용자의 코드가 있는 폴더(`~/d/...`)를 컨테이너의 `/workspace` 에 연결합니다.
- 컨테이너가 삭제되어도 코드는 안전하며, 코드를 수정하면 컨테이너에도 즉시 반영됩니다.

3. 시스템 적용 방법 (Setup)

3-1. 파일 생성

프로젝트 폴더에 다음 두 파일을 생성합니다.

 **Dockerfile (환경 정의서)**

```
# Dockerfile (RTX 5090 / Blackwell 'All-in-One' Edition)
```

```
FROM nvidia/cuda:12.8.0-runtime-ubuntu24.04
```

```
ENV DEBIAN_FRONTEND=noninteractive
```

```
ENV PYTHONUNBUFFERED=1
```

```
ENV PIP_BREAK_SYSTEM_PACKAGES=1
```

```
# 1. 필수 시스템 패키지 설치
```

```
# [수정됨] Ubuntu 24.04에서는 libgl1-mesa-glx 대신 libgl1을 사용해야 합니다.
```

```
RUN apt-get update && apt-get install -y \  
    python3.12 python3-pip python3-dev \  
    git wget curl vim htop nvidia-top \  
    libgl1 libglib2.0-0 \  
    && rm -rf /var/lib/apt/lists/*
```

```
# pip 업그레이드 (충돌 방지 옵션 포함)
```

```
RUN pip3 install --upgrade --ignore-installed pip setuptools wheel
```

```
# 2. [핵심] Blackwell(sm_120) 지원 PyTorch 설치
```

```
# (Docker 캐싱 덕분에 이전에 빌드했다면 이 단계는 0초 만에 스킵됩니다)
```

```
RUN pip3 install --no-cache-dir \  
    torch torchvision torchaudio \  
    --index-url https://download.pytorch.org/whl/cu128
```

```
# 3. [AI/Data Science 풀옵션 설치]
```

```
RUN pip3 install --no-cache-dir \  
    # [LLM & Deep Learning Core]  
    transformers accelerate bitsandbytes sentencepiece \  
    protobuf huggingface_hub datasets \  
    peft trl safetensors tokenizers \  
    \  
    # [Data Analysis & Math]  
    numpy pandas scipy scikit-learn joblib \  
    \  
    # [Visualization]  
    matplotlib seaborn plotly \  
    \  
    # [Image Processing / CV]  
    # (주의: 서버/Docker환경에서는 반드시 headless 버전을 써야 에러가 안 납니다)  
    opencv-python-headless pillow \  
    \  
    # [Development & Utility]  
    tqdm jupyter jupyterlab ipywidgets \  
    \  
    # [Optional: JupyterLab Theme & Extensions]  
    jupyterlab-theme-dark-ocean jupyterlab-extensions
```

```
pytest black isort pyyaml h5py requests ollama
```

WORKDIR /workspace

RUN ln -s /usr/bin/python3 /usr/bin/python

CMD ["/bin/bash"]

docker-compose.yml (실행 설정서)

```
services:
  blackwell-env:
    build: .
    image: blackwell-torch:latest
    container_name: rtx5090-container

    deploy:
      resources:
        reservations:
          devices:
            - driver: nvidia
              count: all
              capabilities: [gpu]

    volumes:
      # [중요] 사용자의 작업 경로를 여기에 지정
      - /home/yongsang/d/[01]Lab_Activity/자율제조과제:/workspace
      - ~/.cache/huggingface:/root/.cache/huggingface

    stdin_open: true
    tty: true
    network_mode: host
    environment:
      - NVIDIA_VISIBLE_DEVICES=all
      - LOG_LEVEL=INFO

    restart: unless-stopped
    command: sleep infinity
```

3-2. 실행 명령어

터미널에서 파일이 있는 경로로 이동하여 실행합니다.

1. 이미지 빌드 (최초 1회 필수, 라이브러리 추가 시 사용)

```
docker compose build
```

2. 컨테이너 실행 (백그라운드)

```
docker compose up -d
```

3. 컨테이너 접속 (작업 시작)

```
docker exec -it rtx5090-container bash
```

4. 라이브러리 관리 및 추가 방법

상황 A: "잠깐 테스트로 깔아보고 싶을 때" (비영구적)

컨테이너 내부에서 그냥 설치하면 됩니다.

```
# 컨테이너 안에서
```

```
pip install some-new-library
```

- **주의:** 컨테이너를 제거(down)하고 다시 만들면 사라집니다.

상황 B: "앞으로 계속 쓸 라이브러리일 때" (영구적 - 권장)

Dockerfile 에 추가하고 이미지를 갱신합니다. PyTorch 등 무거운 건 다시 받지 않으므로 매우 빠릅니다.

1. Dockerfile 수정:

```
# ... 기존 라이브러리들 ...
```

```
pytest black isort pyyaml h5py requests ollama [여기에 추가]
```

2. 재빌드 및 적용 (호스트에서 실행):

```
docker compose build # 변경된 부분만 설치됨 (수 초~수 분 소요)
```

```
docker compose up -d # 컨테이너 교체 (1초 소요)
```

5. 워크스페이스(폴더) 변경 방법

프로젝트가 바뀌어서 연결할 폴더를 바꾸고 싶을 때, **절대로 이미지를 다시 빌드하지 마십시오.**

방법 A: 같은 상위 폴더 내 이동 (가장 추천)

현재 설정이 `/home/yongsang/d/.../자율제조과제` 로 되어 있습니다.

만약 자율제조과제 폴더 안에 `Project_A`, `Project_B` 가 있다면:

1. 그냥 접속 (`docker exec ...`)
2. `cd /workspace/Project_B` 로 이동. 끝.

방법 B: 완전히 다른 경로로 변경

예: `/home/yongsang/e/New_Project` 로 바꾸고 싶을 때.

1. `docker-compose.yml` 수정:

```
volumes:  
  - /home/yongsang/e/New_Project:/workspace # 경로만 수정
```

2. 설정 적용 (1초 소요):

```
# build 명령어 금지! (시간 낭비임)  
docker compose up -d
```

- Docker가 볼륨 설정 변경을 감지하고 컨테이너만 빠르게 재시작합니다.

6. 검증 (Verification)

컨테이너 접속 후 항상 다음 스크립트로 JIT Free 환경인지 확인하십시오.

```
# verify_arch.py
import torch
print(f"GPU: {torch.cuda.get_device_name(0)}")
print(f"Arch List: {torch.cuda.get_arch_list()}")
# 결과에 'sm_120'이 포함되어 있어야 JIT 없이 즉시 실행됩니다.
```